

QuickKart Analytics Assessment

Marketplace & Logistics Take-Home Exercise

Time guidance: Please spend no more than **2–3 hours**. We care more about how you think, explain trade-offs, and prioritize than about polishing every detail. If you cannot fully complete every task, include notes on what you would do next.

1. Business Context

QuickKart is a two-sided e-commerce marketplace operating across multiple Indian cities. Customers place orders with independent sellers; QuickKart coordinates payments and logistics through several carrier partners.

QuickKart promises “**Delivery in 2 days**” on eligible orders. Recently:

- On-time delivery rate has been declining.
- Customer complaints about late deliveries are rising.
- Leadership is concerned about the impact on repeat purchases and GMV.

You are the lead analyst asked to help two stakeholders understand the situation:

Head of Marketplace — cares about GMV, take-rate, category mix, seller performance, and repeat purchases.

Head of Logistics — cares about delivery times, on-time rate, carrier performance, lane efficiency, and cost.

2. Business Questions

Your analysis should address the following:

1. How are GMV and orders trending over time by city and category?
2. How do delivery delays impact repeat purchase rate and cancellation/return rates?
3. Which combinations of seller, city, and carrier are driving the most delays and lost GMV?
4. If we tighten or relax promised delivery times on specific lanes, what could be the impact on GMV and repeat rate?

3. Dataset Overview

You are provided with six CSV files. The data covers approximately 18 months of marketplace activity.

customers.csv

Columns: customer_id, signup_date, city, state, segment

25,000 customers across 12 cities. Segment: Budget, Value, or Premium.

sellers.csv

Columns: seller_id, seller_name, primary_city, rating

400 sellers with ratings 1–5.

products.csv

Columns: product_id, category, subcategory, base_price

3,000 products in 5 categories: Electronics, Fashion, Grocery, Home & Kitchen, Books.

orders.csv

Columns: order_id, customer_id, created_at, status, payment_method, promised_delivery_date, is_fast_delivery_eligible

100,000 orders. Status: Delivered, Shipped, Cancelled, Returned.

order_items.csv

Columns: order_item_id, order_id, product_id, seller_id, quantity, unit_price, discount_pct, platform_fee_pct

~170K line items. platform_fee_pct is QuickKart's take-rate on each line.

shipments.csv

Columns: shipment_id, order_id, carrier, shipped_at, delivered_at, ship_from_city, ship_to_city, shipping_cost, delivery_status

~92K shipments. Carriers: BlueDart, Delhivery, Ekart, InHouse. delivery_status: OnTime, Late_1_2d, Late_3_5d, Late_5p, Lost.

Key Definitions

GMV = sum of (quantity × unit_price) across order items, before platform fee.

Delayed order = delivered_at > promised_delivery_date. Delay buckets are pre-computed in delivery_status.

Repeat customer = any customer with ≥2 delivered orders.

Fast delivery eligible: promised_delivery_date = created_at + 2 days.

4. Tasks & Deliverables

Focus your effort on the parts you find most impactful. Prioritization is part of the evaluation.

A. Exploratory Analysis & Insights

Using SQL and/or Python (pandas, Polars, etc.):

1. Compute and explore:
 - Monthly GMV by city and category.
 - Monthly count of orders and unique active customers.
 - Monthly repeat purchase rate (customers with ≥ 2 delivered orders so far).
 - Share of delayed orders by city and carrier.
2. In a short write-up (max ~1 page in README or markdown):
 - Highlight 3–5 key findings you think leadership should know (e.g., segments, lanes, or carriers that are problematic or promising).
 - Call out any assumptions you made where the data was ambiguous.

B. SQL Questions

Write SQL queries (Postgres-flavored is fine) for the following. Include them in a .sql file or notebook.

B1. Monthly Marketplace Metrics

Return, by calendar month and customer city: month, city, GMV, number_of_orders, unique_customers, repeat_purchase_rate (customers with ≥ 2 delivered orders up to that month / active customers in that month).

B2. Impact of First-Order Delay on Repeat

For customers whose first delivered order was delayed (any Late bucket), compare their 90-day repeat rate vs. customers whose first order was OnTime. Return a table with two rows: first_order_delay_status = 'OnTime' vs 'Delayed', and the corresponding 90-day repeat rate.

B3. Seller–Carrier Performance

For (seller, carrier, ship_to_city) combinations with ≥ 100 delivered orders, return: seller_id, carrier, ship_to_city, total_GMV, delayed_GMV, delayed_order_rate, avg_delay_days (for delayed orders).

B4. Query Optimization

Consider the following naive query:

```
SELECT o.order_id,
       (SELECT delivered_at
        FROM shipments s
        WHERE s.order_id = o.order_id) AS delivered_at,
       o.created_at,
       c.city
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE EXISTS (
  SELECT 1
  FROM shipments s2
```

```
WHERE s2.order_id = o.order_id
AND s2.delivery_status <> 'OnTime'
);
```

- Rewrite this query so it scales better on large tables.
- Explain why your version is more efficient.
- What indexes would you add to support your queries, and how would they help?

C. Dashboard / Mini Web App

Using Streamlit, Plotly Dash, or another lightweight Python-based tool, build a simple interactive app for the Head of Marketplace and Head of Logistics to explore how delivery performance affects GMV and repeat purchases by city, category, and carrier. Assume they have 10 minutes to use it before a meeting.

Required controls:

- Date range picker on orders.created_at.
- Multi-select filters for city, category, and carrier.
- Metric selector: GMV, orders, repeat_rate, delayed_order_rate.

Required views:

- KPI strip: overall GMV, delayed_order_rate, repeat_rate for current filters.
- Time-series chart of selected metric by month.
- Breakdown chart (bar) of selected metric by city or carrier (toggle).
- Short insight section: 3–5 bullets interpreting the view in business terms.

You may pre-aggregate in SQL and load CSVs, or connect directly—your choice. Please include simple instructions on how to run the app.

D. Use of AI Tools

You may use AI tools (e.g., for boilerplate code or SQL syntax). However:

- Add a short section in your README describing where you used AI (files/sections) and how you verified the outputs were correct.
- We will focus in the live follow-up interview on your understanding and ability to adapt your solution.

5. What to Submit

- **README** (markdown): your approach, key assumptions, main findings and recommendations, AI usage.
- **SQL file** (.sql or notebook) with your answers to Section B.
- **App code**: directory with a minimal Streamlit/Dash app and run instructions.
- **Optional**: any notebooks or scripts used for EDA.

Evaluation note: This take-home is the starting point for a live discussion. We will ask you to walk through your work, explain decisions, modify queries on the fly, and discuss business trade-offs. Focus on demonstrating clear thinking over cosmetic polish.